

CENTRO UNIVERSITÁRIO SENAC – SANTO AMARO
ENGENHARIA DE COMPUTAÇÃO – 9º SEMESTRE

Daniel Tsuyoshi Izumi
Marcelo Alfonso de Souza

P2
LINGUAGENS FORMAIS E COMPILADORES

São Paulo
2026

1. INTRODUÇÃO

Este relatório documenta o desenvolvimento de um compilador funcional para a ArithLang, uma mini-linguagem aritmética de escopo reduzido. O objetivo do projeto foi implementar um *pipeline* capaz de transformar código-fonte em texto puro até a execução em uma máquina virtual (VM) de pilha.

O compilador é organizado em cinco fases independentes e sequenciais. A análise léxica lê o código-fonte caractere a caractere e produz uma lista de tokens anotados com linha e coluna. A análise sintática consome essa lista e constrói uma árvore sintática abstrata (AST) por meio de um *parser* de descida recursiva. A análise semântica percorre a AST verificando regras que a gramática não consegue expressar (uso de variáveis não declaradas, redeclaração e atribuição sem declaração prévia) e realiza inferência de tipos. O gerador de código traduz a AST em *bytecode* para uma VM por meio de uma travessia em pós-ordem. Por fim, a máquina virtual executa esse *bytecode* instrução a instrução em um laço de despacho.

A ArithLang suporta números inteiros e de ponto flutuante, as quatro operações aritméticas básicas com precedência convencional, agrupamento por parênteses, declaração de variáveis com *let*, atribuição, comentários de linha iniciados por *#* e o comando *print*. Todas as fases produzem mensagens de erro localizadas com número de linha e coluna, de forma que o compilador seja bem informativo.

2. GRAMÁTICA FORMAL

A gramática da ArithLang foi definida em notação EBNF (*Extended Backus-Naur Form*), onde “*” indica zero ou mais repetições, “+” uma ou mais, “?” zero ou uma ocorrência, “[]” representa alternativa e elementos entre aspas simples são terminais literais.

program → statement* EOF

statement → letStmt
 | assignStmt
 | printStmt

```

letStmt    → 'let' IDENTIFIER '=' expression
assignStmt → IDENTIFIER '=' expression
printStmt  → 'print' '(' expression ')'

expression → term ( ( '+' | '-' ) term )*
term       → unary ( ( '*' | '/' ) unary )*
unary      → '-' unary | primary
primary    → INTEGER
            | FLOAT
            | IDENTIFIER
            | '(' expression ')'

```

Um programa é uma sequência de zero ou mais comandos seguida do marcador de fim de arquivo. Há três formas de comando: *letStmt*, que declara uma nova variável e atribui um valor inicial; *assignStmt*, que atualiza o valor de uma variável previamente declarada; e *printStmt*, que avalia uma expressão e exibe o resultado na saída padrão. As duas primeiras formas se distinguem sintaticamente pela presença ou ausência da palavra-chave *let*, o *parser* identifica qual aplicar ao observar o token seguinte ao identificador.

A hierarquia de *expression*, *term*, *unary* e *primary* encoda diretamente a precedência dos operadores sem nenhuma lógica adicional. Como *term* é chamado por *expression* e *unary* é chamado por *term*, os operadores *** e */* ficam obrigatoriamente em níveis mais profundos da árvore que *+* e *-*, o que garante que sejam avaliados primeiro. O operador unário *-* é tratado por uma regra recursiva à direita, permitindo negações encadeadas. Parênteses são tratados em *primary* e sobrepõem qualquer precedência natural ao forçar uma chamada direta a *expression*.

Os terminais *NEWLINE* e *SEMICOLON* são filtrados pelo *parser* antes da análise e funcionam como separadores implícitos entre comandos, sem aparecer na gramática formal. Isso permite que o programador use tanto quebras de linha quanto ponto-e-vírgula para separar instruções, ou uma combinação dos dois.

3. ANÁLISE LÉXICA

A classe *Lexer* opera sobre o código-fonte como uma string contínua,

mantendo três atributos de estado ao longo da tokenização: *pos*, o índice do caractere atual; *line*, o número da linha atual; e *col*, o número da coluna atual. A cada chamada a *_advance()*, esses três atributos são atualizados de forma consistente: quando o caractere consumido é uma quebra de linha, *line* é incrementado e *col* é resetado para 1; caso contrário, apenas *col* avança. Essa disciplina garante que todos os tokens produzidos carregam a posição exata de onde começam no fonte.

A interface pública é o método *tokenize()*, que executa um laço principal e retorna a lista completa de tokens ao fim do arquivo. A cada iteração, espaços, tabulações e retornos de carro são descartados por *_skip_whitespace()* antes de qualquer decisão de reconhecimento. O caractere atual determina qual caminho seguir: “#” aciona *_skip_comment()*, que avança até o fim da linha sem produzir nenhum token; um dígito aciona *_read_number()*; uma letra ou sublinhado aciona *_read_identifier()*; e os demais caracteres são verificados contra uma tabela de tokens simples de um único caractere.

O reconhecimento de números distingue inteiros de *floats* durante a própria leitura. *_read_number()* consome dígitos e pontos decimais em sequência, usando uma *flag is_float* que é ativada na primeira ocorrência de “.”. Se um segundo ponto for encontrado antes do fim do número, um *LexerError* é lançado imediatamente. Ao final, o token produzido é *INTEGER* com valor Python *int* ou *FLOAT* com valor Python *float*, conforme o caso.

O reconhecimento de identificadores e palavras-chave ocorre em *_read_identifier()*, que consome letras, dígitos e sublinhados enquanto disponíveis. Após a leitura, o lexema é consultado em um dicionário fixo de palavras reservadas e o tipo do token é determinado pelo resultado dessa consulta: *LET* ou *PRINT* para palavras-chave, *IDENTIFIER* para qualquer outro nome.

Qualquer caractere que não se enquadre em nenhuma das categorias mencionadas anteriormente provoca o lançamento de *LexerError*. Os tokens *NEWLINE* e *SEMICOLON* são produzidos pelo *lexer* mas descartados pelo *parser* antes da análise sintática, onde funcionam apenas como separadores implícitos entre comandos.

4. ANÁLISE SINTÁTICA

A classe *Parser* consome a lista de tokens produzida pelo *Lexer* para construir

a árvore sintática abstrata (AST). Antes de iniciar a análise, os tokens do tipo *NEWLINE* e *SEMICOLON* são filtrados da lista, pois cumprem papel apenas de separadores superficiais e não carregam significado estrutural para a gramática. O *parser* mantém um único atributo de estado, *_pos*, que aponta para o token atual na lista filtrada.

A navegação sobre os tokens é feita por três métodos internos: *_advance()* consome e retorna o token atual, avançando *_pos*, exceto quando o token é *EOF*, caso em que a posição não avança para evitar leitura além do fim da lista; *_expect()* verifica se o token atual é do tipo esperado — em caso positivo, chama *_advance()* e retorna o token — caso contrário, lança *ParseError* com a linha do token inesperado e uma mensagem descrevendo o que era esperado e o que foi encontrado; *_match()* apenas verifica o tipo do token atual sem consumi-lo, sendo usado nas decisões de ramificação.

O método de entrada é *parse()*, que chama *_statement()* repetidamente até encontrar *EOF* e retorna um nó *Program* contendo a lista de comandos. A identificação do tipo de comando em *_statement()* segue uma ordem determinística: se o token atual é *LET*, delega para *_let_statement()*; se é *PRINT*, delega para *_print_statement()*; se é *IDENTIFIER* e o token seguinte é *ASSIGN*, delega para *_assign_statement()*. Qualquer outra combinação resulta em *ParseError*. A verificação antecipada do token seguinte ao *IDENTIFIER*, feita via *_peek()* sem consumir nenhum token é o mecanismo que permite distinguir uma atribuição de um início de expressão inválido.

A precedência dos operadores é implementada pela própria cadeia de chamadas entre os métodos de expressão, sem nenhuma tabela de precedência explícita. O método *_expression()* chama *_term()* para obter o operando esquerdo e então consome operadores “+” e “-” em laço, chamando *_term()* novamente a cada iteração. A função *_term()* faz o mesmo com “*” e “/”, chamando *_unary()*. Como *_term()* é chamado por *_expression()* e *_unary()* é chamado por *_term()*, os operadores de maior precedência ficam em níveis mais profundos da árvore e são, portanto, avaliados primeiro. Em *_unary()*, trata-se a negação unária de forma recursiva à direita antes de delegar para *_primary()*, que reconhece os elementos folha: literais inteiros, literais *float*, identificadores e subexpressões entre parênteses.

Todos os nós da AST são definidos em *ast_nodes.py* como classes imutáveis, o que impede modificações acidentais após a construção da árvore. Para ilustrar como

a precedência se manifesta na estrutura da árvore, considere a expressão $2 + 3 * 4$. O *parser* produz a seguinte AST:

```
BinaryOp('+')
├─ IntLiteral(2)
└─ BinaryOp('*')
    ├─ IntLiteral(3)
    └─ IntLiteral(4)
```

O nó `BinaryOp("*")` está um nível abaixo de `BinaryOp("+")`, o que faz com que a multiplicação seja avaliada antes da adição durante a travessia da árvore. Com parênteses forçando a precedência inversa, $(2 + 3) * 4$, a estrutura se inverte:

```
BinaryOp('*')
├─ BinaryOp('+')
│   ├─ IntLiteral(2)
│   └─ IntLiteral(3)
└─ IntLiteral(4)
```

Vale destacar que o padrão *Visitor* é usado para percorrer a AST, ou seja, o despacho é feito dinamicamente: o método `visit()` deriva o nome do método a ser chamado a partir do tipo do nó (`visit_` + nome da classe) e o invoca via `getattr()`. Isso permite que cada fase implemente apenas os métodos correspondentes aos nós que precisa tratar.

5. ANÁLISE SEMÂNTICA

A análise semântica é implementada na classe *SemanticAnalyzer*, que estende *Visitor* e percorre a AST produzida pelo *parser* verificando regras que a gramática não consegue expressar. A fase realiza duas responsabilidades distintas e complementares: verificação de uso correto de variáveis e inferência de tipos. Ao final da travessia, o analisador expõe dois atributos públicos: *table*, a tabela de símbolos populada durante a análise, e *types*, um dicionário que mapeia o identificador de cada nó de expressão ao seu tipo inferido.

A tabela de símbolos é implementada na classe *SymbolTable* como um dicionário $\{ nome: tipo \}$ de escopo global único, refletindo o fato de que a *ArithLang* não possui escopos aninhados. Ela expõe três operações: *define()* registra uma nova variável — se o nome já existir na tabela, lança *SemanticError* com uma mensagem de que a variável já foi declarada anteriormente; *assign()* atualiza o tipo de uma variável existente — se o nome não estiver na tabela, lança *SemanticError* orientando o uso de *let* para a declaração inicial; e *lookup()* consulta o tipo de uma variável — se o nome não estiver na tabela, lança *SemanticError* com uma mensagem de que a variável ainda não foi declarada.

A ordem de visita da AST é determinante para que o uso antes da declaração seja detectado corretamente. Em *visit_LetStatement()*, a expressão do lado direito é visitada primeiro e só então *define()* é chamado para registrar a nova variável. Isso garante que uma expressão como *let b = a + 1* falhe com *SemanticError* se *a* ainda não foi declarado, mesmo que *a* apareça em uma linha posterior do programa.

A inferência de tipos opera sobre os nós de expressão e é propagada de baixo para cima na árvore. Literais *IntLiteral* recebem tipo *int* e literais *FloatLiteral* recebem tipo *float*. Identificadores recebem o tipo registrado na tabela de símbolos no momento da consulta. O operador unário propaga o tipo do operando sem alteração. Para operações binárias, o tipo resultante é determinado conforme as regras abaixo:

Operação	Tipo do operando esquerdo	Tipo do operando direito	Tipo resultante
+, -, *	int	int	int
+, -, *	float	float	float
+, -, *	int	float	float
+, -, *	float	int	float
/	qualquer	qualquer	float

O tipo inferido de cada nó de expressão é armazenado em *types* via *_record()*, que usa o identificador de objeto Python (*id(node)*) como chave. Esse dicionário é produzido para uso potencial por fases posteriores, embora o gerador de código atual não precise dele: as regras de coerção de tipo em tempo de execução são tratadas nativamente pelo Python na máquina virtual.

6. GERAÇÃO DE CÓDIGO E VM

A geração de código e a execução deste são duas etapas do mesmo módulo, o *codegen.py*. A primeira fase traduz a *AST* em uma lista de instruções para uma máquina virtual de pilha, enquanto a segunda executa essa lista instrução a instrução.

O gerador é implementado na classe *CodeGenerator*, que estende *Visitor* e percorre a *AST* em pós-ordem. Cada nó visitado chama o método interno *_emit()*, que serializa a instrução como uma tupla de strings e a acrescenta à lista interna *_instructions*. O método de entrada é *generate()*, que inicia a travessia a partir do nó *Program* e acrescenta a instrução *HALT* ao final, sinalizando à *VM* que a execução deve ser encerrada.

A travessia em pós-ordem é a consequência direta da estrutura dos métodos de visita: em *visit_BinaryOp()*, os filhos esquerdo e direito são visitados antes de emitir o *opcode* da operação. Isso assegura que os operandos estejam no topo da pilha no momento em que a instrução de cálculo for executada, que é o requisito fundamental de uma máquina de pilha. O mesmo princípio se aplica a *visit_UnaryOp()*, que visita o operando antes de emitir *NEG*, e a *visit_LetStatement()* e *visit_AssignStatement()*, que visitam a expressão do lado direito antes de emitir *STORE*.

A tabela a seguir descreve o ISA (*Instruction Set Architecture*) completo, ou seja, o conjunto de instruções que a máquina virtual é capaz de executar:

Instrução	Argumento	Efeito
PUSH	valor	Empurra o literal valor no topo da pilha
LOAD	nome	Busca o valor da variável nome no ambiente e empurra na pilha
STORE	nome	Desempilha o topo e armazena na variável nome do ambiente
ADD	—	Desempilha dois valores e empurra a soma
SUB	—	Desempilha dois valores e empurra a diferença
MUL	—	Desempilha dois valores e empurra o produto
DIV	—	Desempilha dois valores e empurra o quociente; lança erro se o divisor for zero
NEG	—	Desempilha um valor e empurra seu inverso aditivo
PRINT	—	Desempilha o topo e exhibe na saída padrão
HALT	—	Encerra a execução da máquina virtual

Para ilustrar a tradução, considere o programa *let x = 2 + 3 * 4*, cuja *AST* já construímos previamente. A travessia em pós-ordem produz o seguinte *bytecode*:


```
PUSH 2
PUSH 3
PUSH 4
MUL
ADD
STORE x
HALT
```

A multiplicação precede a adição não por qualquer lógica de precedência no gerador, mas porque o nó *BinaryOp('*')* é filho de *BinaryOp('+')* na AST e, portanto, visitado primeiro. O resultado de *MUL* é empurrado na pilha antes de *ADD* ser executado, produzindo o valor correto 14.

O *bytecode* gerado pode ser persistido em disco no formato *.alc* via *save_bytecode()*, que serializa cada instrução como uma linha de texto com o *opcode* seguido de seu argumento separado por espaço. O arquivo pode ser carregado posteriormente via *load_bytecode()* e executado diretamente pela VM, sem necessidade de recompilar o fonte.

A VM é implementada na classe *VirtualMachine* com dois atributos de estado: *stack*, a pilha de operandos, e *env*, o ambiente de variáveis representado como um dicionário *{ nome: valor }*. Um terceiro atributo, *output*, acumula as linhas produzidas por *PRINT* para facilitar verificações nos testes automatizados.

A execução ocorre em *run()*, um laço de despacho que itera sobre a lista de instruções e seleciona o tratamento adequado para cada *opcode* via cadeia de condicionais. As instruções aritméticas (*ADD*, *SUB*, *MUL*, *DIV*) desempilham dois valores na ordem inversa de empilhamento (*b* primeiro, depois *a*) para preservar a associatividade correta da operação. A instrução *DIV* verifica explicitamente se o divisor é zero antes de operar e lança *RuntimeError_* caso seja.

A instrução *PUSH* recebe seu argumento como string e o converte para *int* ou *float* em tempo de execução. A instrução *PRINT* aplica uma regra de formatação de saída: valores *float* cujo valor numérico é igual ao seu equivalente inteiro são exibidos sem casas decimais (15.0 é impresso como 15), enquanto os demais são exibidos com a representação padrão do Python.

7. TESTES

A *suite* de testes é desenvolvida em *test_compiler.py* com o módulo *unittest* da biblioteca padrão do Python. Organizada em quatro classes, ela cobre 28 casos de teste, sendo que todos passaram:

#	Classe	Descrição	Entrada	Saída esperada	Resultado
01	TestLexer	Lexer reconhece INTEGER corretamente	42	Token do tipo INTEGER com valor 42	Ok
02	TestLexer	Lexer reconhece FLOAT corretamente	3.14	Token do tipo FLOAT com valor 3.14	Ok
03	TestLexer	Lexer distingue LET, PRINT e IDENTIFIER	let print x	Três tipos distintos na lista	Ok
04	TestLexer	Comentários são completamente ignorados	# tudo isso some\nlet x = 1	Nenhum token com conteúdo do comentário	Ok
05	TestLexer	Linha e coluna são registrados corretamente	let x = 1\nlet y = 2	Segundo LET na linha 2, coluna 1	Ok
06	TestLexer	LexerError para caractere não reconhecido	let x = @	LexerError com L1 e @ na mensagem	Ok
07	TestParser	Parser constrói LetStatement corretamente	let x = 10	Nó LetStatement com name = "x"	Ok
08	TestParser	Parser distingue AssignStatement de LetStatement	let x = 1\nx = 2	Segundo comando é AssignStatement	Ok
09	TestParser	Multiplicação tem precedência maior que adição na AST	let x = 2 + 3 * 4	Raiz +, filho direito *	Ok
10	TestParser	Parênteses invertem a precedência na AST	let y = (2 + 3) * 4	Raiz *, filho esquerdo +	Ok
11	TestParser	ParseError ao omitir parêntese de fechamento	print(1 + 2	ParseError	Ok
12	TestParser	ParseError ao usar print sem parênteses	print x	ParseError	Ok
13	TestSemantic	SemanticError ao usar variável não declarada	print(y)	SemanticError com 'y' na mensagem	Ok
14	TestSemantic	SemanticError ao redeclarar variável com let	let x = 1\nlet x = 2	SemanticError com 'x' na mensagem	Ok
15	TestSemantic	SemanticError ao atribuir sem declaração prévia	z = 10	SemanticError	Ok
16	TestSemantic	SemanticError ao usar variável antes de declará-la	let b = a + 1\nlet a = 5	SemanticError	Ok
17	TestSemantic	Divisão entre inteiros resulta em tipo float	let a = 10\nlet b = 4\nlet r = a / b	r com tipo "float" na tabela	Ok
18	TestSemantic	Soma int + float resulta em float por promoção	let x = 2\nlet y = 1.5\nlet z = x + y	z com tipo "float" na tabela	Ok
19	TestExecution	As quatro operações produzem saídas corretas	let a = 10\nlet b = 3\nprint(a + b)\n...	13, 7, 30, 3.3333333333333335	Ok

20	TestExecution	Resultados numéricos respeitam precedência	let x = 2 + 3 * 4\nlet y = (2 + 3) * 4\n...	14, 20	Ok
21	TestExecution	Operações com floats produzem resultado correto	let base = 7.5\nlet altura = 4.0\n...	15	Ok
22	TestExecution	Operador unário negativo funciona corretamente	let n = -10\nprint(n)	-10	Ok
23	TestExecution	Atribuição acumulada incrementa variável corretamente	let x = 1\nx = x + 1\nx = x + 1\nprint(x)	3	Ok
24	TestExecution	Promoção int + float retorna resultado float	let i = 3\nlet f = 0.5\nprint(i + f)	3.5	Ok
25	TestExecution	Divisão por zero lança RuntimeError	let a = 5\nlet b = 0\nprint(a / b)	RuntimeError_ com "divisão por zero"	Ok
26	TestExecution	Múltiplos prints produzem saídas na ordem correta	let a = 1\nlet b = 2\nlet c = 3\nprint(a)\n...	["1", "2", "3"]	Ok
27	TestExecution	Expressões profundamente aninhadas são avaliadas corretamente	let r = (2 + 3) * (4 - 1) / 3\nprint(r)	5	Ok
28	TestExecution	Ponto-e-vírgula é aceito como separador de comandos	let x = 10; let y = 20; print(x + y)	30	Ok

8. CONCLUSÃO

O desenvolvimento do compilador apresentou desafios distribuídos ao longo de todas as fases, mas dois deles chamam mais atenção por envolverem decisões com consequências amplas. O primeiro foi garantir que mensagens de erro fossem precisas e localizadas em todas as fases, o que exigiu disciplina no rastreamento de linha e coluna desde o *lexer*, propagando essa informação por todos os nós da AST que pudessem originar um erro semântico. O segundo foi a definição do contrato entre as fases: como cada módulo deveria expor seus resultados para o seguinte sem criar acoplamento desnecessário. A solução foi fazer com que cada fase retorne uma estrutura de dados bem definida, permitindo que qualquer fase seja testada ou substituída independentemente das demais.

Entre as limitações atuais, a mais relevante é a ausência de escopos aninhados: todas as variáveis compartilham um único escopo global, o que impede a reutilização de nomes em blocos distintos e tornaria inviável a implementação de funções. A linguagem também não oferece estruturas de controle, como condicionais e laços, nem tipos compostos, como *strings* ou listas, o que restringe os programas a sequências lineares de operações aritméticas. Por fim, a máquina virtual de pilha

executa o *bytecode* interpretando tuplas em Python, sem nenhuma otimização, o que a torna inadequada para programas com alto volume de cálculo.

As extensões mais impactantes seriam a introdução de um sistema de escopos com pilha de ambientes para suportar funções e variáveis locais, a adição de estruturas de controle com os respectivos nós na AST e instruções de desvio condicional na VM (*JUMP*, *JUMP_IF_FALSE*). Uma melhoria mais ambiciosa seria substituir a máquina virtual por um *backend* de geração de código para uma arquitetura real, como x86-64 ou WebAssembly, eliminando a camada de interpretação e produzindo executáveis nativos.